About: Mock Testing Environment

Introduction of Purpose:

The last couple user stories I have completed have been to the benefit of testing. Our code was coupled in ways not obvious until unit testing began. The biggest issue was relegating object construction to classes that have other responsibilities. For example, the DeviceContext class is responsible for managing and exposing the state of the IBDC device. This class was also responsible for building the components it needed in order to do so. This created a coupling between what the class is supposed to do and how it was supposed to do it. This class is only one example of the what-how coupling, and other files in the project had the same issue.

Why re-factor the code?

As I went to write unit tests, I found out immediately that this would not be an easy task. I wanted to write unit tests for the ImageIngestService file, but ImageIngestService created its own components. This is an issue because the created components rely on native code targeted for iOS and Android operating systems, which are not available at the development/code level we are operating the tests on. This led the team to a decision, find a way to run all the tests natively on the target systems or somehow de-couple the code from the hardware. This is where I made the decision to re-factor.

What code was re-factored and how did it help with unit testing?

The way I decided what code to re-factor was based on hardware boundaries. I chose to re-factor any code that directly interfaces hardware. I think this is a good strategy because it de-couples what something needs to provide from how it’s done. From here, I re-factored the code in every file where our code interfaced hardware. This re-factoring was done by removing object construction from files that specify what needs to be done and passing parameters to these files. The parameters are the constructed objects that were removed from the file. This type of re-factoring was done all the way to the root or starting point of the program. With this design decision, each component of the program can be passed its production parameter or their testing equivalents. This allows to unit testing at the typescript level for most communication within our program.

How are these tests extensible and re-creatable?

The way that the re-factoring was done allows to mock components to be passed in during testing. I created class IBDCCommsToAppEndpointProvider. This class automatically creates all mock components necessary for each testable component in the following modules:

- ImageIngestService - responsible for re-creating images from bytes and writing to the database
- DeviceContext - responsible for maintaining IBDC device state and exposing state to components
- IBDCCommunicationService - responsible for sending and receiving messages to and from the IBDC device and other modules

IBDCCommsToAppEndpointProvider provides a default environment for testing the above components as of now, but eventually it will be a fully customizable testing tool that will enable programmers to define their own parameters to pass into each component, allowing for testing of the modules with different sets of data.

Testing Limitations: Testing at this level is important to make sure written code transforms data as expected. However, this code will eventually be compiled into a different programming language and run in a different and ever changing mobile environment. Thorough use of this testing environment can help programmers determine whether an error is introduced through written code.